

SWINBURNE UNIVERSITY OF TECHNOLOGY

DEPLOYMENT PORTFOLIO TASK 1

Student: Sidhaarth Krishnan

Student ID: 104089870

Unit: SWE40006 Software Deployment and Evolution

Teaching period: Semester 2, 2026

Submission date: 17 August 2026

Declared target: Task 1.4 - High Distinction

Source repository: https://github.com/RealSid08/SWE40006_Task1

Distribution links: [GitHub Release](#) | [WinGet PR #418824](#)

Submission statement: Tasks 1.1 through 1.4 were completed on Windows. The work covers the applications, WiX authoring, validated MSI builds, installation and execution evidence, the required final Hello World uninstall, multiple acquired runtime dependencies for Distinction, and a public WinGet distribution submission for High Distinction. HashGuard Desktop remains installed and available on the PC.

1. Executive summary

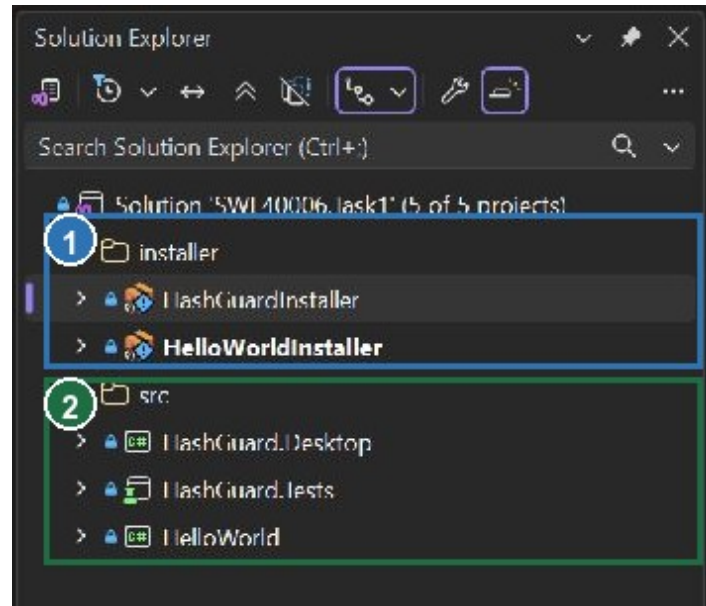
This report documents an end-to-end Windows deployment and distribution workflow using WiX Toolset 4.0.6, .NET 10, GitHub Releases, and WinGet. Tasks 1.1 and 1.2 cover the Hello World and HashGuard applications; Task 1.3 packages the acquired Newtonsoft.Json, CsvHelper, and Humanizer dependencies; and Task 1.4 publishes the final MSI and submits its manifest to the official Microsoft WinGet repository. Hello World was installed, executed, and then uninstalled as required.

Outcome: Final verification produced 4 passing tests and both MSI projects built with 0 warnings and 0 errors. The GitHub Release asset was published with a verified SHA-256 digest, the WinGet manifest passed local validation, and pull request #418824 was submitted to microsoft/winget-pkgs.

1.1 Setup environment and toolchain

Host environment		Native Windows 11 Home, 64-bit, build 26200
VM details		Not applicable - the assignment was completed directly on a Windows PC rather than a macOS virtual machine
Solution		SWE40006.Task1.sln containing five application, test, and installer projects
Build target		Release configuration; win-x64 application publish and x64 MSI packages
Component	Installed version	Role
Visual Studio	Community 2026 18.9.0	.NET desktop workload and solution/project authoring
.NET	SDK 10.0.400; Desktop Runtime 10.0.11	Build, test, publish, and execute the applications
WiX Toolset	CLI and SDK 4.0.6	Compile the two x64 MSI packages
HeatWave	Visual Studio extension installed	WiX project templates and Visual Studio integration

1.2 Visual Studio project evidence



- 1 Installer solution folder containing HashGuardInstaller and HelloWorldInstaller.
- 2 Source solution folder containing HashGuard.Desktop, HashGuard.Tests, and HelloWorld.

Figure 1. Annotated Visual Studio Solution Explorer showing the two WiX installer projects and the three application/test projects.

1.3 Step-by-step setup and build workflow

1. **Set up the Windows toolchain:** The environment used Visual Studio Community 2026 18.9.0 with the .NET desktop development tools, .NET SDK 10.0.400, .NET Desktop Runtime 10.0.11, WiX Toolset 4.0.6, and the HeatWave Visual Studio extension.
2. **Open and verify the solution:** SWE40006.Task1.sln was opened in Visual Studio, where HelloWorld, HashGuard.Desktop, HashGuard.Tests, HelloWorldInstaller, and HashGuardInstaller were confirmed as loaded.

3. **Restore, build, and test the applications:** Pinned NuGet packages were restored, the C# projects were built in Release mode, and the HashGuard test project was run. The final test result was four passes with no failures or skipped tests.
4. **Publish the deployable application files:** Hello World and HashGuard were published for win-x64 as framework-dependent applications. This produced a stable Release payload for each installer containing the executable, application DLL, dependency manifest, and runtime configuration.
5. **Author the WiX packages:** The .wxs files define the installation directories, application files, product identity, upgrade behaviour, and shortcuts. Task 1.3 adds separate WiX components for Newtonsoft.Json.dll, CsvHelper.dll, and Humanizer.dll.
6. **Build both MSI packages:** HelloWorldInstaller and HashGuardInstaller were compiled in Release mode. Both final builds produced the expected x64 MSI file with 0 warnings and 0 errors.
7. **Install and inspect the deployed files:** Each MSI was installed with Windows Installer verbose logging, followed by exit-status and installed-directory checks. Both installation logs ended with status 0, and the expected files were present.
8. **Execute and verify the installed applications:** Hello World was run from its installed directory and its banner was captured before the package was uninstalled. The installed HashGuard application was launched to calculate both checksums for the sample payload and confirm that a history record was saved. Final verification confirmed that the Hello World product registration and application directory were removed while HashGuard remained installed.

1.4 Console proof of successful builds and installs

Final Release build output - evidence/31 and evidence/32

```

HelloWorldInstaller -> ...\\SWE40006-HelloWorld-1.0.0-x64.msi
Build succeeded.
    0 Warning(s)
    0 Error(s)

HashGuardInstaller -> ...\\HashGuard-Desktop-1.0.0-x64.msi
Build succeeded.
    0 Warning(s)
    0 Error(s)

```

Final Windows Installer log excerpts - evidence/43 and evidence/44

```

Windows Installer installed the product. Product Name: SWE40006 Hello World.
Installation success or error status: 0.
MainEngineThread is returning 0

Windows Installer installed the product. Product Name: HashGuard Desktop.
Installation success or error status: 0.
MainEngineThread is returning 0

```

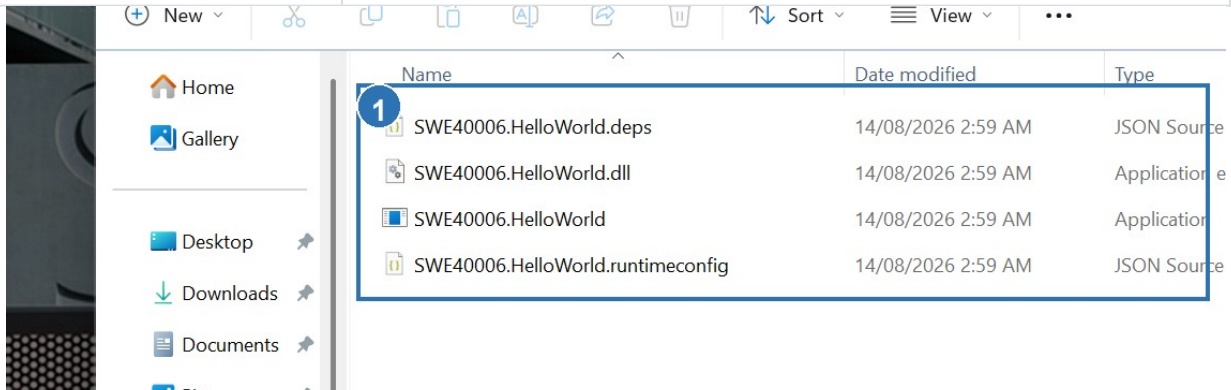
1.5 Reproducibility controls

WiX SDK and extension versions are pinned in the .wixproj files, and the NuGet dependency versions are locked. Stable product and upgrade GUIDs provide consistent package identity. Final test, build, installation, execution, and metadata logs are retained in the evidence directory, while both MSI SHA-256 hashes are recorded in evidence/33-msi-metadata.txt.

2. Task 1.1 - Pass: Hello World MSI

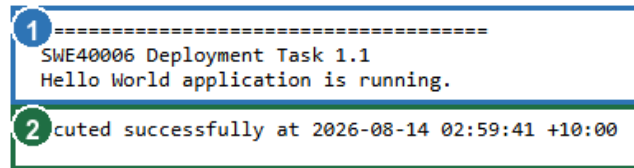
The Pass task uses a small .NET console program that prints an assignment-specific banner. A WiX x64 package installs the published executable and its three companion runtime files to %LOCALAPPDATA%\Programs\SWE40006 Hello World. The MSI was installed silently, the deployed executable was run from that directory, and the output was captured. The package was then uninstalled as required; Windows Installer returned status 0, and final verification confirmed that its product registration and application directory were removed.

MSI	SWE40006-HelloWorld-1.0.0-x64.msi
Product version	1.0.0
Product code	{40C76FDB-606D-4DE6-9B17-F06149341B0A}
Install result	Exit code 0; four application files present
Execution result	Installed application printed the expected Task 1.1 banner
Final state	Package uninstalled; product registration and application directory removed



- 1 The installed directory contains the Hello World executable, application DLL, dependency manifest, and runtime configuration.

Figure 2. Annotated installed-directory evidence for the Task 1.1 Hello World package.



The image shows a console window with two lines of output. The first line is enclosed in a blue box and has a blue circle with the number '1' to its left. The second line is enclosed in a green box and has a green circle with the number '2' to its left.

```
1 -----  
SWE40006 Deployment Task 1.1  
Hello World application is running.  
2 cuted successfully at 2026-08-14 02:59:41 +10:00
```

- 1 The installed executable printed the assignment-specific Task 1.1 banner.
- 2 The captured process completed successfully with exit code 0.

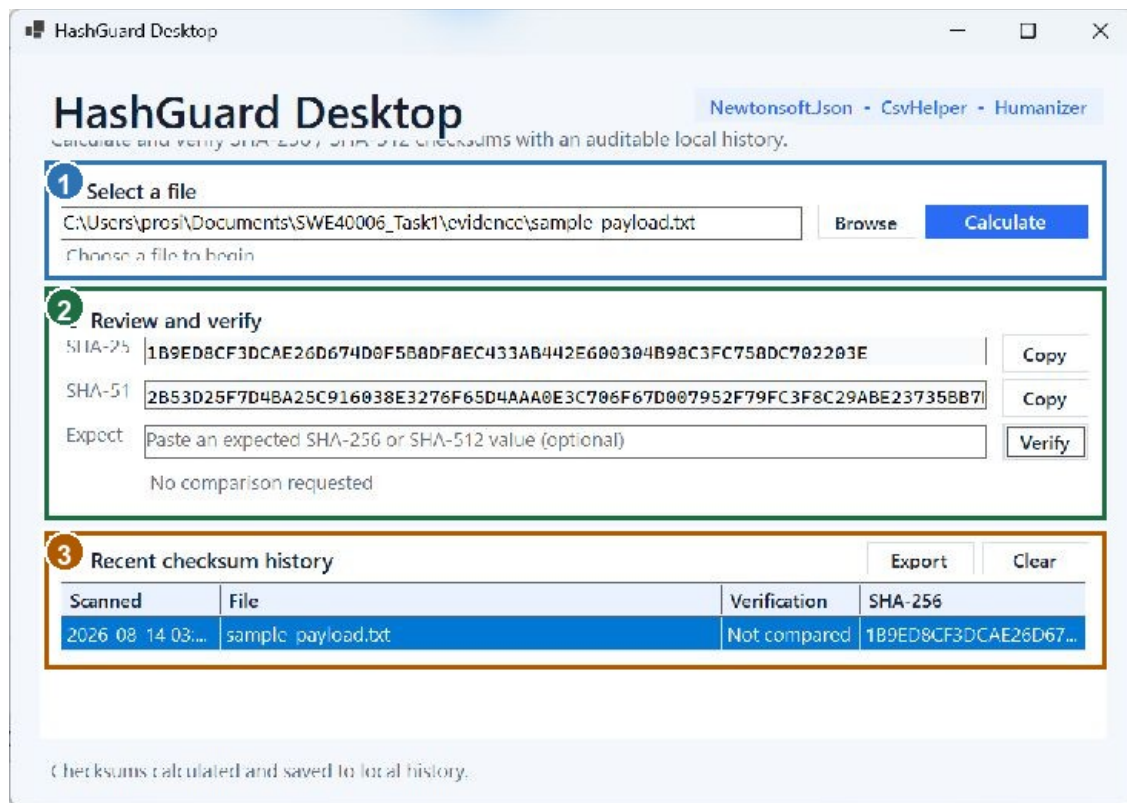
Figure 3. Annotated console output from the installed Hello World executable.

3. Task 1.2 - Credit: HashGuard Desktop

The Credit task introduces HashGuard Desktop, an original C# WinForms application for calculating SHA-256 and SHA-512 file checksums. Its implementation includes asynchronous hashing, optional comparison with an expected hash, a local JSON audit history, and CSV export. The interface also identifies the external dependencies and displays complete hash values for copying and verification.

3.1 Application design

Layer	Implementation	Responsibility
Presentation	MainForm.cs	File selection, progress, hash display, comparison, history, and export actions
Hashing	HashService.cs	Buffered asynchronous SHA-256/SHA-512 computation using IncrementalHash
Persistence	HistoryStore.cs	Auditable local JSON history using Newtonsoft.Json
Export	CsvExportService.cs	CSV output through CsvHelper
Presentation helper	Humanizer	Readable dates and display text



- 1 Input file: evidence/sample-payload.txt.
- 2 HashGuard calculated and displayed complete SHA-256 and SHA-512 values.
- 3 The application stored the completed calculation in its local audit history.

Figure 4. Annotated execution evidence from the installed HashGuard application.

3.2 Core checksum implementation

C# excerpt - HashService.ComputeAsync

```
using var sha256 = IncrementalHash.CreateHash(HashAlgorithmName.SHA256);
using var sha512 = IncrementalHash.CreateHash(HashAlgorithmName.SHA512);
await using var stream = new FileStream(
    filePath,
    FileMode.Open,
    FileAccess.Read,
    FileShare.Read,
    BufferSize,
    FileOptions.Asynchronous | FileOptions.SequentialScan);

var buffer = new byte[BufferSize];
long totalRead = 0;
int bytesRead;

while ((bytesRead = await stream.ReadAsync(buffer, cancellationToken)) > 0)
{
    sha256.AppendData(buffer, 0, bytesRead);
    sha512.AppendData(buffer, 0, bytesRead);
    totalRead += bytesRead;
    progress?.Report(fileInfo.Length == 0 ? 100 : (int)(totalRead * 100 / fileInfo.Length));
}

progress?.Report(100);
return new HashResult(
    fileInfo.FullName,
    fileInfo.Length,
    Convert.ToHexString(sha256.GetHashAndReset()),
    Convert.ToHexString(sha512.GetHashAndReset()));
```

HashService streams each file in 1 MiB buffers and appends every buffer to both hash algorithms. This avoids loading the entire file into memory and allows the interface to display progress for large inputs.

3.3 Automated verification

```
HashGuard.Desktop -> C:\Users\prosi\Documents\SWE40006_Task1\src\HashGuard.Desktop\bin\Release\net10.0-windows\HashGuard.Desktop.dll
HashGuard.Tests -> C:\Users\prosi\Documents\SWE40006_Task1\src\HashGuard.Tests\bin\Release\net10.0-windows\HashGuard.Tests.dll
Test run for C:\Users\prosi\Documents\SWE40006_Task1\src\HashGuard.Tests\bin\Release\net10.0-windows\HashGuard.Tests.dll (.NETCoreApp,Version=v10.0)
A total of 1 test files matched the specified pattern.
1 .d! - Failed: 0, Passed: 4, Skipped: 0, Total: 4, Duration: 68 ms - HashGuard.Tests.dll (net10.0)
```

1 The final Release test run completed with 4 passed, 0 failed, and 0 skipped tests.

Figure 5. Annotated console output from the final automated test run.

Test	Expected behaviour	Result
Known hashes	SHA-256 and SHA-512 match known values	Passed
Mismatch verification	Malformed or different expected hash is rejected	Passed
JSON round trip	Newtonsoft.Json persists and restores a history record	Passed
CSV export	CsvHelper writes an export containing the history data	Passed

4. Task 1.3 - Distinction: external dependencies

Distinction criterion: The Distinction criterion is addressed by authoring multiple runtime dependencies as individual WiX components. The final HashGuard installation was verified to contain Newtonsoft.Json.dll, CsvHelper.dll, and Humanizer.dll alongside the application.

4.1 Runtime dependency set

Dependency	Pinned version	Use in HashGuard
Newtonsoft.Json	13.0.4	Serializes and deserializes checksum history
CsvHelper	33.1.0	Exports the audit history to CSV
Humanizer.Core	3.0.10	Formats readable time and display values

4.2 WiX authoring

WiX v4 excerpt - explicit external DLL components

```
<Component Id="NewtonsoftJsonComponent" Guid="{9AAAABB3-2199-4AFB-B3F9-14AAAB88A66A}">
  <File Id="NewtonsoftJsonDll" Source="..\..\src\HashGuard.Desktop\bin\Release\net10.0-windows\win-
x64\publish\Newtonsoft.Json.dll" />
  <RegistryValue Root="HKCU" Key="Software\SidharthKrishnan\HashGuardDesktop\Components"
Name="NewtonsoftJson" Type="integer" Value="1" KeyPath="yes" />
</Component>
<Component Id="CsvHelperComponent" Guid="{F6EBD07F-2291-4527-9317-4739766DD50B}">
  <File Id="CsvHelperDll" Source="..\..\src\HashGuard.Desktop\bin\Release\net10.0-windows\win-
x64\publish\CsvHelper.dll" />
  <RegistryValue Root="HKCU" Key="Software\SidharthKrishnan\HashGuardDesktop\Components"
Name="CsvHelper" Type="integer" Value="1" KeyPath="yes" />
</Component>
<Component Id="HumanizerComponent" Guid="{EA9E4267-5AA4-4E47-9AAA-463335F372FD}">
  <File Id="HumanizerDll" Source="..\..\src\HashGuard.Desktop\bin\Release\net10.0-windows\win-
x64\publish\Humanizer.dll" />
  <RegistryValue Root="HKCU" Key="Software\SidharthKrishnan\HashGuardDesktop\Components"
Name="Humanizer" Type="integer" Value="1" KeyPath="yes" />
</Component>
```

Each acquired DLL has its own stable component GUID, while an HKCU registry value supplies the per-user component key path required by full MSI validation. The same package also defines the application executable, application DLL, dependency manifest, runtime configuration, installation directory, cleanup components, and Start Menu shortcut.

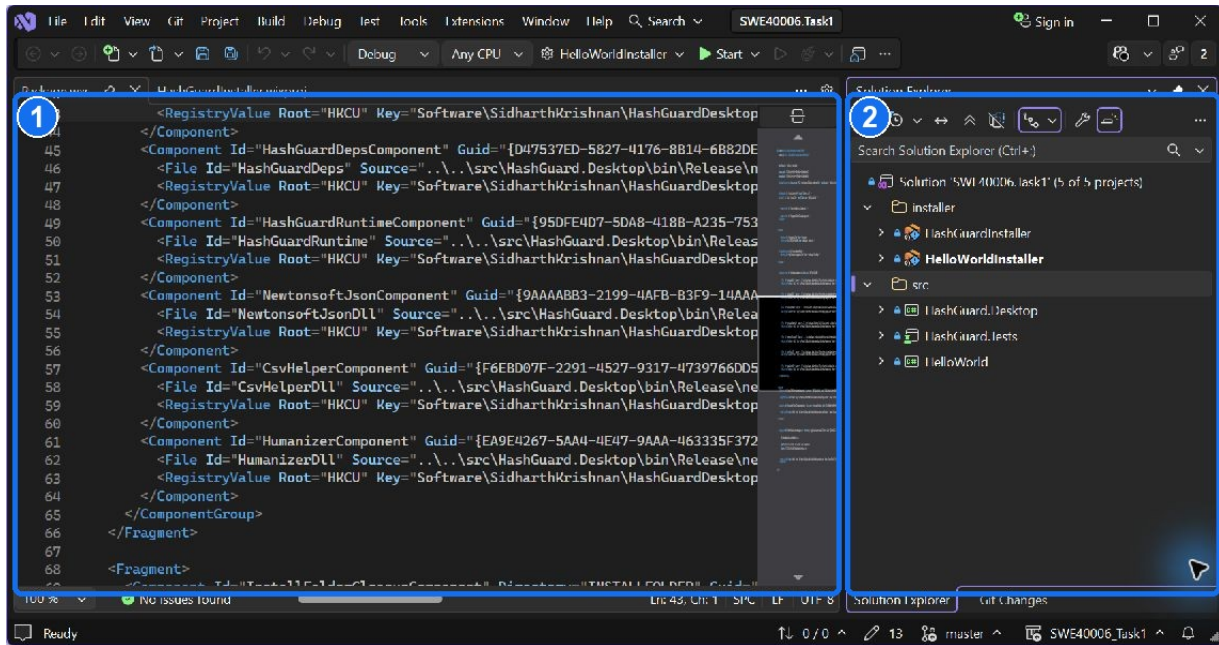
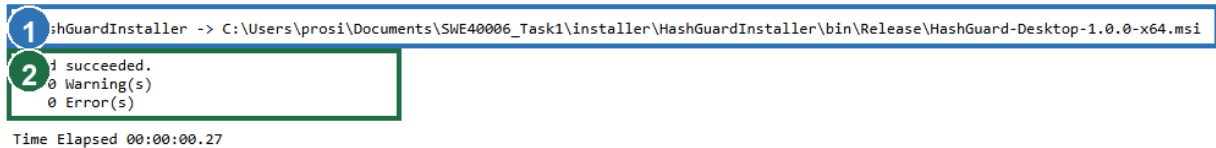


Figure 6. Final Visual Studio view of Package.wxs after the full-validation corrections.

Annotations: 1 shows the acquired DLL components with HKCU registry key paths; 2 confirms the complete five-project solution in Visual Studio.

4.3 Build and installed-file evidence



- 1 The Release build produced HashGuard-Desktop-1.0.0-x64.msi.
- 2 The WiX build succeeded with 0 warnings and 0 errors.

Figure 7. Annotated console output from the final HashGuard MSI build.

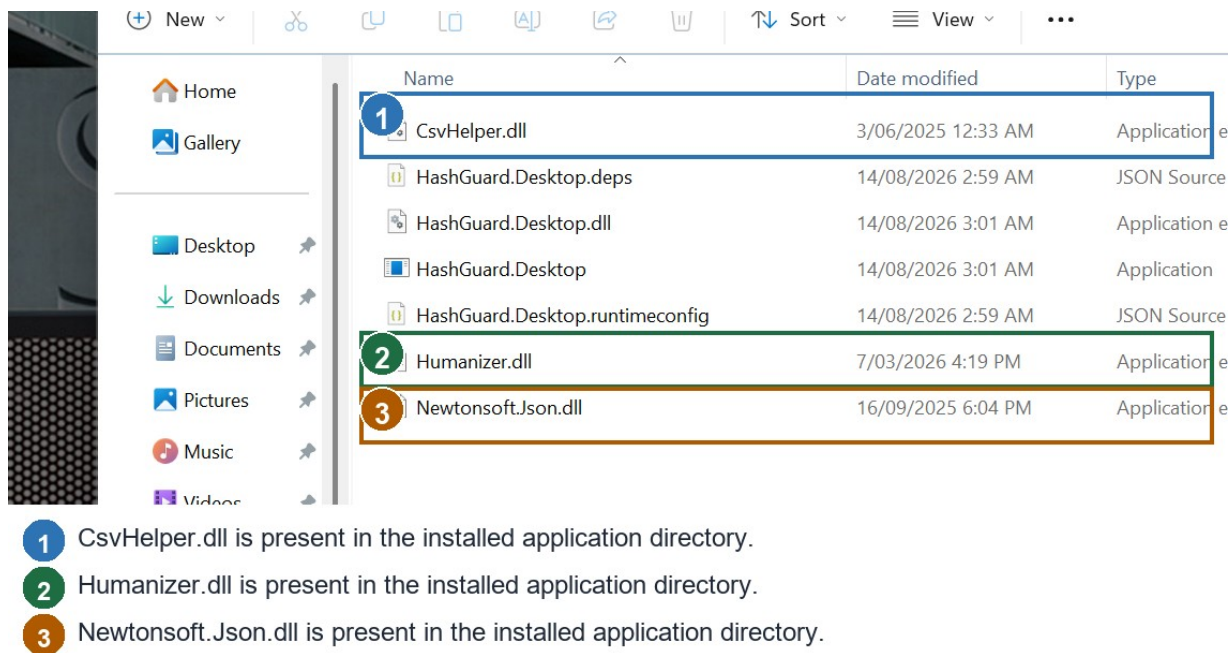


Figure 8. Annotated installed-directory evidence for the three external runtime DLLs.

4.4 MSI identity and installation

MSI	HashGuard-Desktop-1.0.0-x64.msi
Product code	{C57B16A8-E9E5-4EE9-8020-EE069EB409A7}
Upgrade code	{23AB1A31-A44A-41FE-BBDA-5245030BAD76}
Size	1,097,728 bytes
SHA-256	A947612491037D6512AFA6AECEC5838628D7A8B899F6520B476F42FE7AFE6A5B
Final install	Exit code 0; 7 files plus Start Menu shortcut
Execution	Installed application launched and processed evidence/sample-payload.txt

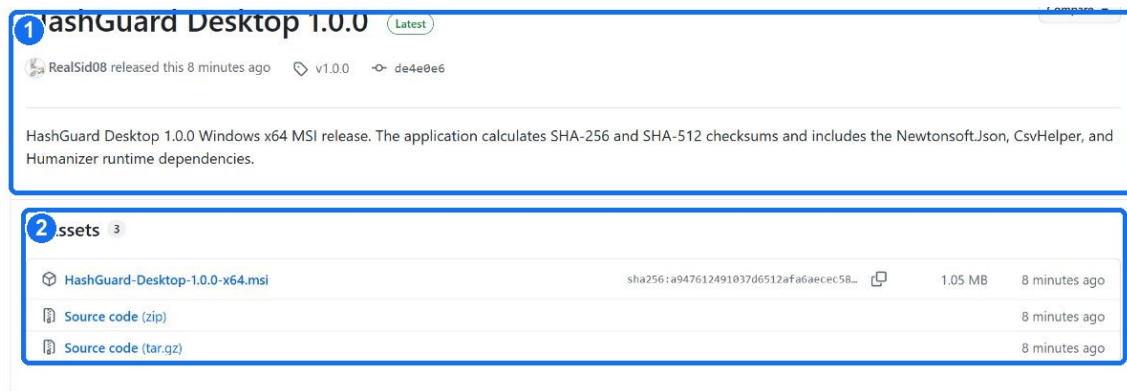
5. Task 1.4 - High Distinction: WinGet distribution

The submitted distribution uses package identifier SidharthKrishnan.HashGuardDesktop and version 1.0.0. The public release, manifest URL, MSI digest, package identity, and repository submission all refer to the same validated installer.

5.1 Release preparation and MSI validation

Before publication, the HashGuard tests were run again, the win-x64 application was published, and the WiX installer was rebuilt with Windows Installer validation enabled. The final build completed with 0 warnings and 0 errors. A direct installation of the release MSI returned exit code 0 and installed the seven expected files under the HashGuard Desktop directory.

Release	https://github.com/RealSid08/SWE40006_Task1/releases/tag/v1.0.0
MSI asset	HashGuard-Desktop-1.0.0-x64.msi
File size	1,097,728 bytes
SHA-256	A947612491037D6512AFA6AECEC5838628D7A8B899F6520B476F42FE7AFE6A5B
Product code	{C57B16A8-E9E5-4EE9-8020-EE069EB409A7}



© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

Figure 9. Public GitHub Release containing the final HashGuard x64 MSI asset.

Annotations: 1 identifies the public v1.0.0 release; 2 identifies the downloadable MSI asset and GitHub digest.

5.2 WinGet manifest authoring and validation

The WinGet submission uses the required three-file layout: a version manifest, an x64 installer manifest, and an en-US default-locale manifest. The installer manifest declares the WiX installer type, user scope, product code, public release URL, exact SHA-256 digest, and Microsoft.DotNet.DesktopRuntime.10 as a package dependency with minimum version 10.0.0.

```
PackageIdentifier: SidharthKrishnan.HashGuardDesktop
PackageVersion: 1.0.0
InstallerType: wix
Scope: user
Dependencies:
  PackageDependencies:
    - PackageIdentifier: Microsoft.DotNet.DesktopRuntime.10
      MinimumVersion: 10.0.0
InstallerUrl: https://github.com/RealSid08/SWE40006_Task1/releases/download/v1.0.0/HashGuard-Desktop-1.0.0-x64.msi
InstallerSha256: A947612491037D6512AFA6AECEC5838628D7A8B899F6520B476F42FE7AFE6A5B

> winget validate --manifest manifests\s\SidharthKrishnan\HashGuardDesktop\1.0.0
Manifest validation succeeded.
```

The dependency identifier was checked independently through WinGet, which resolved Microsoft.DotNet.DesktopRuntime.10 at version 10.0.11. The MSI was downloaded back from the release URL and hashed again; its digest matched the manifest exactly.

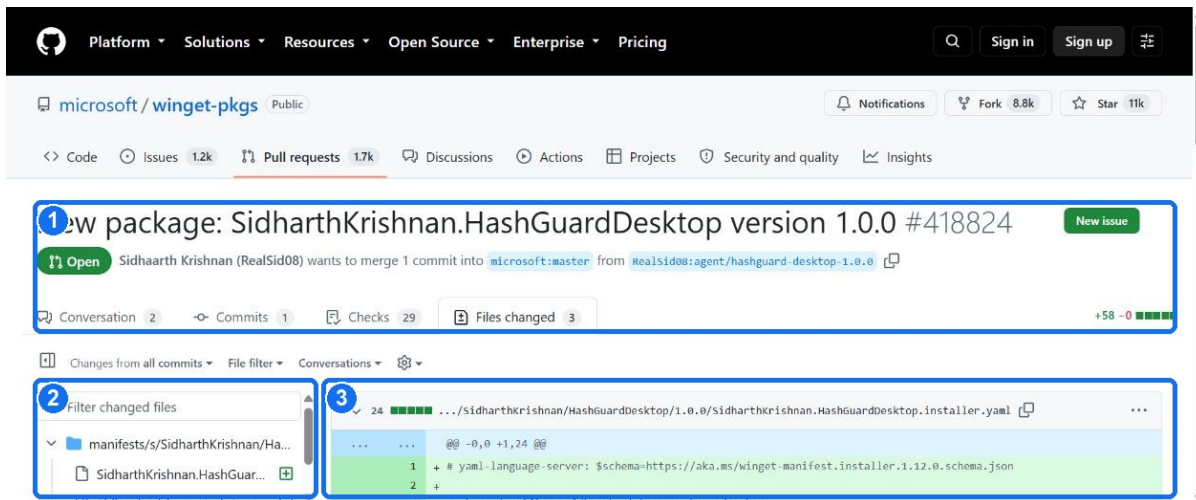


Figure 10. Three WinGet manifest files submitted to the official repository.

Annotations: 1 confirms the official repository and open PR; 2 shows the three-file manifest tree; 3 shows the submitted installer YAML diff.

5.3 Official WinGet repository submission

The manifest was committed on branch `agent/hashguard-desktop-1.0.0` in the `RealSid08/winget-pkgs` fork and submitted to [microsoft/winget-pkgs pull request #418824](#). The pull request is open against `microsoft:master` with one commit, three manifest files, and 58 added lines. The initial pull request, manifest, URL, domain, and manifest-policy checks all passed.

Pull request	https://github.com/microsoft/winget-pkgs/pull/418824
Repository	microsoft/winget-pkgs
State	Open; ready for repository review
Change	3 manifest files; 58 additions
Initial validation	PR, manifest, URL, domain, and policy checks passed

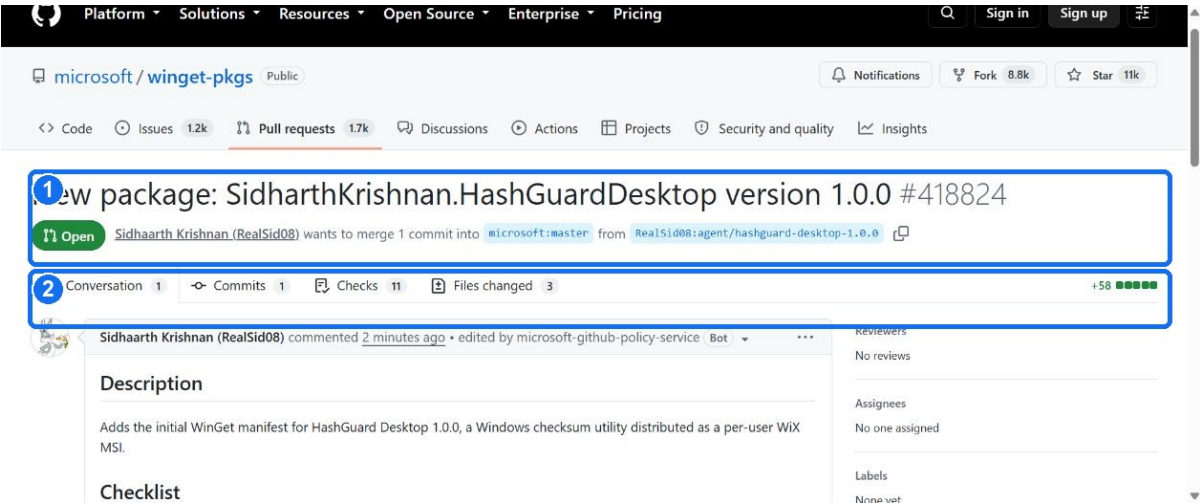


Figure 11. Open package manifest pull request in microsoft/winget-pkgs.

Annotations: 1 identifies the package, PR number, open state, and branch direction; 2 records the commit and three changed manifest files.

6. Manual checksum result

Input	evidence/sample-payload.txt
SHA-256	1B9ED8CF3DCAE26D674D0F5B8DF8EC433AB442E600304B98C3FC758DC702203E
SHA-512	2B53D25F7D4BA25C916038E3276F65D4AAA0E3C706F67D007952F79FC3F8C29AB E23735BB7D6E1A41C3AE4A89D9EE1694A82E1894C4A9EBC3F1B121A640C47AF
History	One record written to the local JSON audit history

7. Troubleshooting and engineering decisions

7.1 WinForms designer serialization warning

Initial compilation produced WFO1000 because WinForms designer serialization interpreted public form state as design-time content. The form was refactored so its application services and state are private readonly fields initialized in the constructor. This separated runtime state from designer serialization and restored a clean build.

7.2 WiX v4 UI migration

The initial WiX build used the WiX v3 UIRef pattern and failed with WIX0094 because that identifier is protected in WiX v4. The package was migrated to the WiX v4 UI extension namespace using ui:WixUI with WixUI_InstallDir and INSTALLFOLDER. WixToolset.UI.wixext is pinned to version 4.0.6.

7.3 Installed-app runtime failure

The first installed HashGuard build terminated before displaying its window. The Windows Application Event Log identified a .NET 10 NotSupportedException caused by a transparent button border setting. Setting BorderSize to 0 corrected the custom styling, after which the tests, publish, MSI build, installation, and installed-application execution all completed successfully.

7.4 Full MSI validation

A full Windows Installer validation build initially reported ICE38 and ICE64 because files under the per-user profile were being used as component key paths. Each file component was changed to use an HKCU registry value as its key path, and cleanup components were added for the installation folders. ICE91 was suppressed because the package is explicitly per-user. The rebuilt MSI then completed full validation with 0 warnings and 0 errors before release.

7.5 Packaging decisions

Decision	Reason	Effect
Per-user MSI	Avoids unnecessary elevation for a student utility	Installs under LocalAppData and supports silent testing
Explicit DLL components	Makes Distinction dependency evidence unambiguous	Each external DLL has a stable component GUID and key path
Framework-dependent publish	Keeps the MSI small while targeting the installed Desktop Runtime	Runtimeconfig and deps files are deployed with the application
GitHub Release asset	Provides a stable public HTTPS installer URL and digest	The exact x64 MSI referenced by WinGet is downloadable from release v1.0.0
Official WinGet PR	Uses Microsoft repository review and validation for package distribution	The three-file manifest was submitted as PR #418824

8. Reproduction procedure

The following commands were run from the repository root on Windows with .NET 10 and WiX Toolset 4.0.6 installed. Pinned project dependencies allow the same commands to reproduce the test, publish, build, and installation results.

PowerShell - build and test

```
dotnet test src/HashGuard.Tests/HashGuard.Tests.csproj -c Release
dotnet publish src/HashGuard.Desktop/HashGuard.Desktop.csproj -c Release -r win-x64 --self-contained false
dotnet build installer/HelloWorldInstaller/HelloWorldInstaller.wixproj -c Release
dotnet build installer/HashGuardInstaller/HashGuardInstaller.wixproj -c Release
```

PowerShell - install

```
msiexec /i installer\HelloWorldInstaller\bin\Release\SWE40006-HelloWorld-1.0.0-x64.msi /qn /L*v evidence\43-restored-hello-install.log
msiexec /i installer\HashGuardInstaller\bin\Release\HashGuard-Desktop-1.0.0-x64.msi /qn /L*v evidence\44-restored-hashguard-install.log
```

PowerShell - final Hello World uninstall

```
msiexec /x {40C76FDB-606D-4DE6-9B17-F06149341B0A} /qn /norestart /L*v evidence\48-final-hello-uninstall.log
```

PowerShell - validate the WinGet manifest and verify the public asset

```
winget validate --manifest distribution\winget\1.0.0
Get-FileHash .\HashGuard-Desktop-1.0.0-x64.msi -Algorithm SHA256
# Expected SHA-256:
# A947612491037D6512AFA6AECEC5838628D7A8B899F6520B476F42FE7AFE6A5B
```

8.1 Expected outputs

Step	Success condition	Final observed result
Test	All automated tests pass	4/4 passed
Build	Both MSI projects succeed	0 warnings; 0 errors
Install	MSI exits 0 and files appear	Both exits 0; 4 and 7 files respectively
Run	Installed executable starts	Hello banner and HashGuard checksum result captured
Uninstall	Hello World MSI exits 0 and installed files are removed	Exit 0; product registration and application directory absent
Release	MSI is publicly downloadable and its digest is fixed	v1.0.0 asset published; 1,097,728 bytes; SHA-256 verified
Distribute	Valid manifest PR is submitted to the official repository	microsoft/winget-pkgs PR #418824 is open with three manifest files

9. References

FireGiant. HeatWave Community Edition and Visual Studio integration. <https://docs.firegiant.com/heatwave/>

FireGiant. WiX Toolset v4 UI extension (WixUI). <https://docs.firegiant.com/wix/tools/wixext/wixui/>

Microsoft. IncrementalHash class. <https://learn.microsoft.com/dotnet/api/system.security.cryptography.incrementalhash>

NuGet. Newtonsoft.Json 13.0.4. <https://www.nuget.org/packages/Newtonsoft.Json/13.0.4>

NuGet. CsvHelper 33.1.0. <https://www.nuget.org/packages/CsvHelper/33.1.0>

NuGet. Humanizer.Core 3.0.10. <https://www.nuget.org/packages/Humanizer.Core/3.0.10>

Swinburne University of Technology. Deployment Portfolio Task 1.

<https://swinburne.instructure.com/courses/77400/assignments/796862>

Microsoft. Windows Package Manager Community Repository. <https://github.com/microsoft/winget-pkgs>

GitHub Release. HashGuard Desktop 1.0.0. https://github.com/RealSid08/SWE40006_Task1/releases/tag/v1.0.0

Microsoft WinGet pull request #418824. <https://github.com/microsoft/winget-pkgs/pull/418824>